

Smpl[•]

FREE GUIDE

Build Your Own *Internal Tools*

A practical guide to replacing SaaS bloat
with tools that actually fit how you work.

Claude Code

Figma

Next.js

Supabase



smpl.as

What's inside *the guide*

We built our own CRM, newsletter tool, task planner, and Meta Ads integration. This guide shares what we learned — what worked, what to avoid, and how to get started.

01 Why Build Your Own?

The real cost of SaaS sprawl — paying for a thousand features and using forty.

02 What's Easy to Build

CRMs, task planners, dashboards — what takes days vs. weeks, and what to avoid.

03 The Stack

Supabase, Next.js, Vercel, and AI coding assistants. What you need and what it costs.

04 Figma for Your Style Guide

Setting up a basic design system — colors, typography, components.

05 Building with Claude Code

How to prompt effectively, what to build first, and how to iterate.

06 What to Watch Out For

Scope creep, maintenance, and other tradeoffs to be aware of.

07 5-Day Getting Started Plan

A practical week-long plan to go from idea to deployed tool.

Why Build *Your Own*?

Most companies run their business across a stack of tools they didn't choose, they inherited. A CRM here, a newsletter tool there, a project planner somewhere else. You end up paying for a thousand features and using forty.

Nothing connects properly. You switch tabs more than you do actual work. And every time you want to change something, the answer is an upgrade, a plugin, or a workaround.

The barrier to building your own tools has dropped enough that most companies could build at least some of their own, the ones where off-the-shelf platforms don't fit.

THE COST OF SAAS SPRAWL

10+

Average SaaS tools
per company

~10%

Features actually
used per tool

30%

Time lost to
tab-switching

That doesn't mean building your own tools is trivial. You still need to think about what you're building and why. But the gap between what generic platforms offer and what your business actually needs is getting easier to close.

What's Actually *Easy to Build*

These used to require a dev team and months of work. With AI coding tools, most take days or weeks, and fit your workflow better than any off-the-shelf platform. (Times are subject to complexity).

Simple CRM

2–3 weeks

Contact tracking, pipeline stages, activity log

Task Planner

1 week

Team scheduling linked to clients or projects

Newsletter Tool

1 week

Templates, send tracking, subscriber management

Internal Dashboard

2–3 days

KPIs, charts, real-time data from your APIs

Landing Page Gen.

1 week

AI-generated pages deployed at custom URLs

Admin Panel

1 week

Manage content, users, or settings for any app

Client Portal

1–2 weeks

Shared project status, files, and communication

API Reporting

1–2 weeks

Pull data from Meta, Google, Stripe — one dashboard

The *Stack*

You don't need a massive tech stack. Four tools handle most of what you need — and they're either free or very cheap to start.

Supabase

Database + Auth

Free → \$25/mo

PostgreSQL database, authentication, and real-time subscriptions. Free tier covers most small tools. Think of it as Firebase but with a real database.

Next.js

Frontend + API

Free

React framework for building the UI and API routes. Server-side rendering, file-based routing, and easy deployment. The backbone of everything.

Vercel

Hosting + Deploy

Free → \$20/mo

One-click deploys from GitHub. Preview URLs for every branch. Custom domains, SSL, and edge functions included.

Claude Code

AI Coding Assistant

\$20/mo

Writes code, debugs, refactors, and explains. Handles 80% of the boilerplate so you can focus on business logic and UX.

Total to get started

\$20/month

Figma for Your *Style Guide*

Your design system isn't just for designers. With Figma MCP, your AI agent can read your styles and build with them.

HOW IT WORKS

Design System in Figma

Set up your colors, typography, spacing, and components in Figma. This becomes your single source of truth.

Connect via Figma MCP

Figma's MCP server lets Claude read your file directly — variables, components, styles. No copy-pasting needed.

Agent Learns Your Brand

The agent understands your design language and applies it consistently when building UI or generating pages.

Design + Code in One Flow

Your agent can search your design system, import components, and build screens that match your brand.

PRO TIP

One of our designers connected her Claude agent to a client's Figma file via MCP. The agent now designs on behalf of the client — pulling real components and applying their brand.

Building with *Claude Code*

AI coding assistants don't replace thinking, but they eliminate most of the boilerplate. Here's how to get the most out of them.

1 Start with the data model

Describe your database tables and relationships first.
The AI builds better code when it understands the structure.

2 Describe what, not how

Say "build a contacts page with search, pagination, and an add-contact modal" — not "create a React component with useState..."

3 Connect your design system

Use Figma MCP so your agent reads your components and styles directly. It'll build UI that matches your brand.

4 Build in layers

Start with data fetching. Then layout. Then interactions.
Don't try to build everything in one prompt.

5 Review and iterate

The first output won't be perfect. Read it, understand it, and ask for specific changes. Each round gets better.

EXAMPLE PROMPT

```
Build a contacts page for my CRM.  
Supabase table: contacts (id, name,  
email, company, status, created_at).  
Use colors: #141416, #C8FF00, #F5F5F0.
```

What to *Watch Out For*

Building your own tools is easier than it used to be. That doesn't mean it's free of tradeoffs. Keep these in mind.



Scope creep is real

Build what you need, nothing more. It's tempting to keep adding features — resist. Ship the minimum useful version first.



Maintenance is on you

There's no support team to call. Keep things simple so they're easy to debug and update. Complexity is the enemy.



Start with one tool, not five

Pick the most painful or wasteful tool in your stack. Build that one first. Prove the concept before expanding.



Security matters

Use proper authentication (Supabase Auth is built in). Never store secrets in client-side code. Use environment variables.



Don't rebuild everything

Payment processing, accounting, compliance tools — these are better left to specialists. Build the gaps, not the foundation.



Iterate, don't perfect

The first version won't be perfect. That's fine. Ship it, use it, and improve it based on real usage.

Get Started in *5 Days*

You don't need to rip out everything at once. Start with one tool — the one that's most painful or most wasteful. Here's a week plan.

Day 1

Audit

What tools do you use? What do you actually need? What's annoying?

Day 2

Pick One

Choose the most painful tool. That's your first build. Don't replace everything at once.

Day 3

Set Up

Supabase + Next.js + Vercel. Design the data model. Get the stack running.

Day 4

Build

Build the core feature. Use AI to move fast. Don't over-think it — functional beats pretty.

Day 5

Ship

Deploy it. Test with your team. Collect feedback. Iterate from real usage.



Want us to *build it for you?*

We've built 125+ digital products for startups, scaleups, and global brands. If you want help building your own internal tools, or want us to do it, we're here.

[Book a Free Consultation](#)

SAY HELLO

Andreas Melvær

Co-founder, SmplCo

andreas@smpl.as

smpl.as · [LinkedIn](#) · [Blog](#)